ShoppingCart — Lab 1: Shopping Cart with Spring Boot and Thymeleaf

Mouad El Goual · DSS — Desarrollo de Sistemas Software Basados en Componentes y Servicios · October 2026
GitHub repository: https://github.com/elgmouad/DSSBCS-ShoppingCart-Spring-

1. Overview

ShoppingCart is a server-rendered web application built with Spring Boot. Visitors browse a product catalogue, filter it by name and price, and manage a shopping cart stored in their HTTP session. Administrators create, edit and delete products and download the catalogue as an SQL script. Access to product management is protected by form-based authentication with roles.

2. Technologies

Area	Technology
Language and build	Java 25, Maven (Maven Wrapper included)
Framework	Spring Boot 4.1.1, Spring MVC
Views	Thymeleaf, Thymeleaf Layout Dialect, Bootstrap 5 and Bootstrap Icons
Security	Spring Security, Thymeleaf Spring Security extras, BCrypt password hashing
Persistence	Spring Data JPA, Hibernate, H2 in-memory database
Other	Jakarta Bean Validation, Lombok, springdoc-openapi (Swagger UI)

3. Architecture

The application follows a layered MVC architecture: each layer only calls the layer below it. Controllers receive requests and select a Thymeleaf view, services hold the business logic, and repositories access the database. Services are defined as interfaces with an implementation, and dependencies are injected through constructors.

Package / folder	Responsibility
`controller`	`HomeController`, `LoginController`, `ProductController`, `CartController`, `AdminController`
`service`	`ProductService`, `CartService` (session-scoped), `DatabaseExportService`, each with an implementation
`repository`	`ProductRepository`, `UserRepository` (Spring Data JPA)
`model`	`Product`, `User` (JPA entities) and the `Role` enum
`config`, `security`	`SecurityConfig` (access rules, login, logout) and `DatabaseUserDetailsService`
`templates`	`layout/base.html`, fragments (navbar, alerts, footer) and the pages `index`, `login`, `productos`, `formulario-producto`, `cart`, `admin`

4. Implemented features

- **Product catalogue (CRUD).** `GET /products` lists products with optional `name`, `minPrice` and `maxPrice` filters. `/products/add`, `/products/edit/{id}`, `/products/save` and `/products/delete/{id}` manage the catalogue. The form validates that the name is not blank and the price is not negative.
- **Shopping cart.** `CartServiceImpl` is a session-scoped bean, so each visitor has an independent cart held in memory. Users can add products, remove a product, empty the cart and see the total (`/cart`, `/cart/add/{id}`, `/cart/remove/{id}`, `/cart/clear`).

- **Authentication and roles.** Users and their roles are stored in the H2 database and loaded by `DatabaseUserDetailsService`; passwords are hashed with BCrypt. A custom login page is served at `/login`, and logout invalidates the session.
- **SQL export.** `GET /admin/export` downloads `products.sql`, a script with one `INSERT` statement per product. `GET /admin` shows an administration page with the product list.
- **User interface.** All pages share a common layout (`layout:decorate`) with reusable fragments. Success and error messages are shown after each action using flash attributes and the Post/Redirect/Get pattern.

5. Security rules

Routes	Access
`/`, `/login`, `GET /products`, `/cart/**`, static resources	Public
`/products/add`, `/products/edit/**`, `/products/save`, `/products/delete/**`	`ADMIN` role
`/admin/**` (administration page and SQL export)	`ADMIN` role
Any other route	Authenticated user

Product management requires the `ADMIN` role, which is stricter than the lab statement (any authenticated user). Managing the catalogue is an administrative task, so an ordinary customer account cannot change it. An authenticated user without the required role is redirected to `/products?accessDenied=true`. CSRF protection is enabled for all forms.

6. Running the application

From the project root, run `./mvnw spring-boot:run` (macOS, Linux) or `.\mvnw.cmd spring-boot:run` (Windows), then open `http://localhost:8080`. On startup the application inserts 12 sample products and two accounts: `admin` / `admin` (roles `ADMIN` and `USER`) and `user` / `user` (role `USER`). The H2 console is available at `/h2-console` (JDBC URL `jdbc:h2:mem:testdb`, user `sa`, empty password). Because the database is in memory, data is reset each time the application stops.